

JMeter & Performance Testing

Load, stress, spike & soak testing, JMeter elements, key metrics, correlation + 25 viva questions with answers

Quick-scan guide for SQA interviews and vivas

Companion to: API Testing Guide · Selenium Guide · Playwright Guide · CI/CD Guide · SQA A-Z Guide

PART 1 — Performance testing basics

Functional testing asks "does it work?" Performance testing asks **"does it still work when 10,000 people use it at once?"** A shopping site that works for 1 user but crashes on flash-sale day has passed functional testing and failed performance testing.

The 5 types (guaranteed viva question)

Type	Question	Everyday picture
Load testing	Does it work at EXPECTED traffic?	Normal Friday evening crowd in a restaurant
Stress testing	Where does it BREAK, and does it fail gracefully?	Keep seating people until the kitchen collapses
Spike testing	Can it survive a SUDDEN jump?	A tour bus of 60 people walks in at once
Soak / endurance	Does it survive load for a LONG time (memory leaks)?	Restaurant full for 72 hours straight
Scalability testing	Does adding servers actually add capacity?	Does opening a second kitchen double the output?

The metrics you measure (memorize all 6)

Metric	Simple meaning
Response time	How long one request takes, start to finish (ms). The user's experience.
Latency	Time until the FIRST byte of the response arrives (network + server start).
Throughput	How many requests the system handles per second — the system's capacity.
Error rate	% of requests that failed. 2% errors at load = thousands of angry users.
Concurrent users	How many virtual users are active at the same moment.
Percentiles (P90/P95/P99)	P95 = 95% of users were at least this fast. Averages LIE — one 30-second outlier hides in a good average, but P95/P99 expose it. Report percentiles, not averages.

Viva favourite: "Why percentiles instead of average?" — Because averages hide the pain. If 99 requests take 1 s and one takes 60 s, the average looks fine (1.6 s) but a real user just waited a minute. P99 shows it.

PART 2 — What is JMeter?

Apache JMeter is a free, open-source, Java-based tool that **simulates many users hitting your system at once**. Each simulated user is a **thread**. Important: JMeter is NOT a browser — it sends protocol-level requests (HTTP, JDBC, FTP, JMS...) and never renders pages or runs JavaScript. That's why one laptop can simulate thousands of users.

The building blocks (know this table cold)

Element	What it is	Example
Test Plan	The root container of everything	'Booking API load test'
Thread Group	The virtual users: how many, ramp-up time, loop count	100 users, ramp-up 60 s, loop 10
Sampler	One actual request	HTTP Request to POST /booking
Config element	Shared settings/data	HTTP Header Manager, CSV Data Set Config

Element	What it is	Example
Assertion	Pass/fail check on the response	Response Assertion: code = 200; Duration Assertion < 2000 ms
Timer	Think time between requests (humans pause!)	Constant/Random timer 1–3 s
Listener	Collects & shows results	Summary Report, Aggregate Report, View Results Tree
Controller	Logic/flow control	Loop, If, Once Only, Transaction Controller
Pre/Post-processor	Runs before/after a sampler	Regex/JSON Extractor to grab a token

Ramp-up explained (always asked)

100 threads with ramp-up 60 s means JMeter starts users gradually — one new user every 0.6 s — instead of hammering all 100 instantly (which would be a spike test, not a load test). Realistic traffic grows gradually.

PART 3 — The techniques that make you sound senior

Parameterization (CSV Data Set Config)

1,000 virtual users must NOT all log in as the same person (caching would fake good results). CSV Data Set Config feeds each thread a different row — username, password, search term — referenced as \${username}. Same idea as the data-driven CSV in my Postman project.

Correlation (extracting dynamic values)

Servers return dynamic values — session IDs, tokens, CSRF values — that the NEXT request must send back. Correlation = capture the value from response 1 with a JSON Extractor or Regular Expression Extractor, store it in a variable, and inject \${token} into request 2. Exactly what I did in Postman with pm.collectionVariables — same concept, different tool. Recorded scripts fail without correlation; fixing that is the core JMeter skill.

Assertions in performance tests

Without assertions, a load test can 'pass' while every response is an error page. Minimum: Response Assertion (status/content) + Duration Assertion (SLA). A fast wrong answer is still wrong.

Running it properly: non-GUI mode

GUI mode is ONLY for building/debugging scripts — it eats memory and distorts results. Real runs use the command line:

```
jmeter -n -t test-plan.jmx -l results.jtl -e -o report-folder
# -n non-GUI · -t plan · -l raw results · -e -o generate HTML dashboard
```

The HTML dashboard gives response-time graphs, percentiles, throughput, and error tables. For very large loads, JMeter runs distributed: one controller machine drives many load-generator machines.

A realistic workflow (your story for the viva)

1. Define the goal with stakeholders: '500 concurrent users, P95 under 2 s, error rate under 1%'.
2. Script the user journey (browse → login → create booking) with realistic think-time timers.
3. Parameterize with CSV, correlate the auth token.
4. Smoke-run with 5 users in GUI to debug, then run the real load non-GUI.
5. Analyze: percentiles, throughput curve, error rate; watch server CPU/memory alongside.

6. Report findings with graphs; rerun after fixes (regression for performance).

PART 4 — 25 Viva Questions with Answers

Q: What is performance testing?

A: Testing speed, stability, and scalability under load — does the system still work when thousands use it at once?

Q: What is JMeter?

A: A free, open-source, Java-based tool that simulates many concurrent users at the protocol level (HTTP and more) to measure performance.

Q: Load vs stress testing?

A: Load = expected traffic, verify SLAs hold. Stress = push beyond capacity to find the breaking point and check graceful failure/recovery.

Q: What is spike testing?

A: Sudden extreme jump in users (flash sale) — can the system absorb it and recover?

Q: What is soak/endurance testing?

A: Sustained load for hours/days to catch slow problems: memory leaks, connection-pool exhaustion, disk fill-up.

Q: What is a thread in JMeter?

A: One virtual user. A Thread Group defines how many users, how fast they start (ramp-up), and how many times they loop.

Q: What is ramp-up period?

A: Time to start all threads. 100 threads / 60 s ramp-up = one new user every 0.6 s — gradual, realistic growth.

Q: What is a sampler?

A: The element that sends one actual request — HTTP Request is the most common.

Q: What is a listener?

A: Collects and displays results — Summary Report, Aggregate Report, View Results Tree (debug only).

Q: Why are timers important?

A: Real humans pause between clicks. Without think-time timers you're testing an unrealistic robot stampede and your numbers mean nothing.

Q: Response time vs latency?

A: Latency = time to FIRST byte; response time = complete response received. Latency isolates network + server start; response time is what the user feels.

Q: What is throughput?

A: Requests handled per unit time (req/s) — the system's capacity. Watch where it plateaus while load keeps rising: that's saturation.

Q: Why report percentiles instead of averages?

A: Averages hide outliers. P95 = 95% of users were at least this fast — it exposes the slow tail that averages bury.

Q: What is parameterization and how in JMeter?

A: Different data per virtual user via CSV Data Set Config, referenced as `${variable}` — prevents cache-skewed results.

Q: What is correlation?

A: Capturing dynamic server values (session ID, token) from one response and injecting them into later requests, using JSON/Regex Extractors. Without it, recorded scripts break.

Q: How do you handle auth tokens in JMeter?

A: Login sampler → JSON Extractor grabs the token → HTTP Header Manager or `#{token}` sends it on later requests — identical concept to my Postman collection-variable chaining.

Q: Why assertions in a load test?

A: Otherwise errors count as 'responses' and the test lies. Response Assertion checks correctness, Duration Assertion enforces the SLA.

Q: Why non-GUI mode for real tests?

A: GUI consumes huge resources and skews results. Build in GUI, run with `jmeter -n -t plan.jmx -l results.jtl`, generate the HTML dashboard with `-e -o`.

Q: What is distributed testing?

A: One controller drives multiple load-generator machines to create more load than a single machine can — needed for very large user counts.

Q: Can JMeter test more than HTTP?

A: Yes: JDBC (databases), FTP, JMS, SOAP, TCP, and more via plugins.

Q: JMeter vs Selenium?

A: Different jobs: Selenium = functional UI, real browser, renders JS, few users. JMeter = performance, protocol level, no rendering, thousands of virtual users.

Q: Is JMeter a browser?

A: No — it never renders pages or executes JavaScript. That's exactly why it can simulate thousands of users from one machine.

Q: What do you monitor on the server during a test?

A: CPU, memory, disk I/O, DB connections, garbage collection — client-side numbers alone can't tell you WHY it's slow.

Q: What would you report after a load test?

A: Test goal & profile, P90/P95/P99 response times, throughput curve, error rate, saturation point, server resource graphs, bottleneck analysis, and recommendations.

Q: How does performance testing fit CI/CD?

A: A small smoke-level load test can run in the pipeline (non-GUI JMeter) with pass/fail thresholds — catching performance regressions early, same philosophy as my Newman CI suite.

Last-minute quick scan

If they say...	You say...
5 types?	Load, stress, spike, soak, scalability.
JMeter?	Java-based protocol-level load tool; thread = virtual user; not a browser.
Ramp-up?	Time to start all threads — gradual, realistic.
Key metrics?	Response time, latency, throughput, error rate, P95/P99.

If they say...	You say...
Average vs P95?	Averages hide the slow tail; percentiles expose it.
Correlation?	Extract dynamic token/session from response → inject into next request.
Real runs?	Always non-GUI: <code>jmeter -n -t plan.jmx -l results.jtl -e -o report</code> .
Assertions?	Response + Duration — fast errors are still errors.

Bridge line for viva: "My Postman suite already asserts response-time SLAs per request; JMeter is the next level — the same endpoints under hundreds of concurrent users, with correlation replacing my collection-variable chaining."